CORE

Claude Code의 핵심 제품 모델

파일·함수 목록이 아니라, 코드 변경을 성립시키는 책임과 불변조건으로 읽기

> **모델의 제안을 안전하고 복구 가능한 저장소 효과로 바꾸는 런타임이 이 프로젝트의 중심이다.**

대상
v2.1.88 / 2ca5ddabfed5

관점
제품 본질 · 책임 · 불변조건

제3자 복원 소스에 대한 교육용 분석이다. 누락된 모듈과 복원 한계를 구현 사실로 보완하지 않는다.

분석 대상을 먼저 제한한다

이 보고서는 공개된 제3자 복원 저장소를 현재 구현의 완전한 원전으로 취급하지 않는다. 존재하는 코드가 보여 주는 설계 압력과 책임 배치를 학습 대상으로 삼는다.

판단의 기준

README가 밝힌 v2.1.88 복원본이며 누락 모듈과 불완전한 빌드가 있다. 따라서 “왜 Anthropic이 그랬는가”가 아니라 “이 코드가 어떤 문제를 닫으려 하는가”를 말한다.

포함

핵심 실행 책임

컨텍스트 구성, 모델 turn, tool 효과, 권한, 세션 정합성, 복구.

포함

개발 압력

레이스·stale write·캐시·취소·호환성·관측 가능성.

보조

UI와 generator

핵심 책임을 전달하고 관찰하게 하는 메커니즘으로만 다룬다.

제외

의도 추정

누락 파일의 내용을 만들어 내거나 조직의 실제 개발 프로세스를 단정하지 않는다.

코드 좌표 README_KR.md · QUICKSTART.md

이 프로젝트는 “대화형 코드 변경 런타임”이다

채팅은 입구다. 제품이 완성해야 하는 일은 불완전한 자연어 요구를 저장소 문맥에 결합하고, 모델의 행동 제안을 검증된 효과로 실행하며, 결과를 다음 추론과 재개 가능한 세션에 되돌리는 것이다.

Context → Proposal → Authorized Effect → Observation → Reconciliation

이 다섯 항 중 하나라도 빠지면 제품은 답변기는 될 수 있어도 코딩 에이전트가 되지 못한다. **Context**가 틀리면 엉뚱한 파일을 고치고, **Effect**가 통제되지 않으면 안전하지 않으며, **Reconciliation**이 없으면 다음 turn과 resume가 거짓 상태를 본다.

핵심 단위는 “메시지”가 아니라 코드 변경 작업	메시지·스트림·UI는 작업의 상태를 운반하는 표현이다. 설계 판단은 최종 저장소 효과와 그 정합성에서 역산해야 한다.

무엇이 핵심인지 판별하는 다섯 질문

파일 수나 기술적 흥미가 아니라 제거했을 때 제품 약속이 무너지는지를 본다.

판별 질문	강한 신호	이 프로젝트의 대표 위치
제품 필수성	없으면 코드 이해·변경·검증이 성립하지 않음	context, query, Read/Edit/Bash
상태 소유권	다음 turn과 resume가 참조하는 정식 상태를 보유	QueryEngine, messages, sessionStorage
효과 권한	파일·프로세스·네트워크에 실제 변화를 발생	Tool.call, permission, sandbox
실패 폐쇄성	취소·fallback·부분 실행 후에도 protocol이 유효	tool pair repair, tombstone, terminal reason
변경 결합도	수정 시 prompt·policy·UI·persistence가 함께 움직임	Tool contract, query loop, REPL/SDK projection

rubric의 용도

점수를 예쁘게 만들기 위한 표가 아니다. UI, MCP, plugin, generator 같은 요소를 제품 본질과 같은 층에 놓는 오류를 막는 필터다.

핵심·필수 지지·확장 경계를 나눈다

모든 기능이 중요하지만 같은 종류로 중요하지는 않다.

등급	책임	판정
핵심	컨텍스트/turn engine/tool effect/permission/session reconciliation	코딩 작업의 의미와 안전성을 직접 소유
필수 지지	REPL·SDK projection, model transport, observability, cost control	핵심을 사용 가능하고 운영 가능하게 만듦
능력 확장	MCP, plugin, skill, subagent, remote bridge	핵심 계약 위에 새로운 공급자·도구·작업자를 얹음
표현 메커니즘	React/Ink, async generator, Zod, AbortSignal	책임을 구현하지만 그 자체가 제품 책임은 아님

영상의 UI·generator 위치

둘은 “큰 축의 일부”다. generator는 turn의 시간 구조를, React/Ink는 동일 상태의 대화형 projection을 구현한다.

“함수 이름을 바꾸고 테스트하라”가 요구하는 것

짧은 요청도 실제로는 읽기·판단·쓰기·검증·회복이 결합된 작업이다.

01	**작업 해석**	명령/skill/일반 prompt를 구분하고 현재 cwd·규칙·선택 영역을 결합한다.
02	**증거 수집**	검색과 읽기로 대상·호출자·테스트를 찾고 read state를 만든다.
03	**변경 제안**	모델이 Edit/Bash tool_use와 구조화된 입력을 생성한다.
04	**권한·정합성**	schema, path, deny rule, read-before-write, stale-write를 검증한다.
05	**효과 실행**	파일을 수정하고 LSP·IDE·history·telemetry를 갱신한다.
06	**재검증·합류**	테스트 결과를 tool_result로 되돌리고 다음 turn 또는 terminal을 결정한다.

결정적인 코드

핵심은 API를 한 번 호출하는 코드가 아니다. “변경 직전에도 읽었던 파일과 같은가”, “실행 결과가 같은 tool_use id로 돌아가는가”, “중간 취소 뒤 resume가 가능한가”를 닫는 코드다.

정식 상태는 한 배열에만 있지 않다

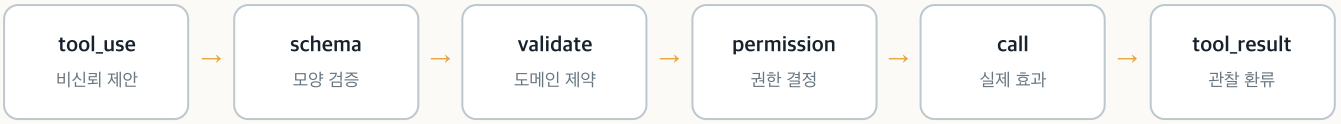
제품 상태는 서로 다른 수명과 신뢰 수준을 가진 여러 저장소에 분산된다. 이를 구분하지 않으면 UI state를 domain state로 오해한다.

상태	소유/수명	지켜야 할 불변조건
conversation messages	turn·conversation	tool_use/result pairing, 순서, compact boundary
ToolUseContext	turn과 하위 tool 실행	permission, abort, read cache, app state 접근
readFileState	conversation	Edit 전 full read와 최신성 확인
AppState/store	interactive session	permission mode, MCP pool, task/UI 상태
transcript JSONL	process 밖의 durable session	append order, parent chain, flush/resume
filesystem/process	외부 세계	부분 효과·사용자 동시 변경·subprocess 수명

코드 좌표 src/QueryEngine.ts · src/Tool.ts · src/utils/sessionStorage.ts

모델 출력은 실행 권한이 아니라 제안이다

모델은 tool 이름과 JSON 입력을 제안한다. 런타임은 존재 확인, schema parsing, 의미 검증, permission, hook, sandbox를 거친 뒤에만 효과를 허용한다.



이 경계는 단순 validation pipeline이 아니다. 실행 전에는 authority를 좁히고, 실행 후에는 결과를 모델이 이해할 protocol message로 변환한다. 따라서 Tool 계약은 모델 API, 보안, runtime, UI를 동시에 잇는 anti-corruption layer다.

코드 좌표 src/services/tools/toolExecution.ts · src/Tool.ts

핵심 난점은 호출 순서보다 시간의 중첩이다

모델 스트림, tool 실행, 사용자 추가 입력, hook, prefetch, transcript write가 동시에 진행된다.

중첩

stream과 tool

tool_use block이 완성되면 모델 응답이 끝나기 전 실행을 시작할 수 있다.

동시성

read 병렬·write 직렬

입력별 isConcurrencySafe 판정으로 순서를 보존한다.

취소

cooperative abort

signal은 요청일 뿐 이미 발생한 외부 효과의 rollback은 아니다.

내구성

write queue와 flush

화면의 완료와 디스크의 완료가 같은 순간이 아니다.

배경지식

이 시스템은 synchronous call stack보다 **structured concurrency**, **optimistic concurrency control**, **saga/compensation** 관점으로 읽을 때 실패 경로가 보인다.

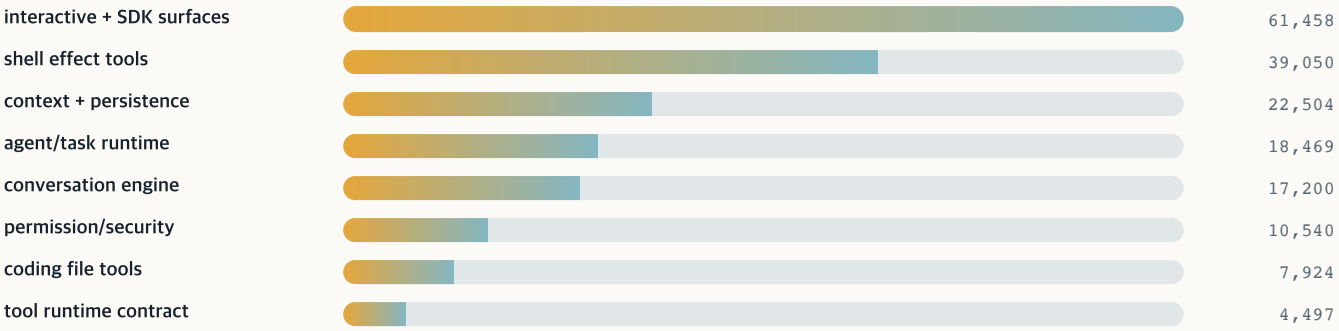
코드를 따라갈 때 먼저 적어야 할 불변조건

흐름도보다 불변조건이 리팩터링과 오류 분석에 오래 남는다.

영역	대표 불변조건	깨질 때 증상
protocol	모든 tool_use에는 같은 id의 tool_result가 존재	다음 API 요청 400, resume 단절
file edit	마지막 full read 이후 외부 변경이 없어야 함	사용자 변경 덮어쓰기
prompt cache	캐시 prefix의 bytes/order가 안정	비용·latency 급증
stream fallback	실패한 attempt의 partial message/effect가 재시도와 섞이지 않음	중복 실행·orphan
session	사용자 입력은 응답 전 crash에도 복구 가능	resume에서 대화 없음
projection	REPL·SDK가 같은 core event를 각 계약에 맞게 투영	표시/저장/API 결과 불일치

코드 분포는 기능 수가 아니라 위험의 위치를 보여 준다

선택한 핵심 주변부만 보아도 shell 효과와 surface/session 쪽에 큰 코드량이 축적된다. 이는 “LLM 호출”보다 외부 세계와의 접점이 더 비싸다는 신호다.



해석 주의

라인 수는 중요도 점수가 아니다. 다만 shell parsing·보안과 session/surface 정합성에 예외 처리와 호환성 코드가 집중되어 있음을 보여 주는 탐색 지표다.

언어 기능은 책임을 지키는 장치로 읽는다

문법 이름을 외우는 대신 어떤 제품 불변조건을 값·타입·제어 흐름으로 옮겼는지 본다.

표현	여기서 맡은 역할	핵심 연결
async generator	progressive event와 terminal reason을 한 실행에서 전달	turn engine / stream / tool progress
discriminated union	message·permission·terminal의 가능한 상태를 닫음	exhaustive handling
Zod schema	모델 JSON을 runtime value로 승인	untrusted proposal → typed input
AbortSignal	model/tool/hook에 취소를 전파	부분 효과와 cleanup
immutable clone	API cache용 원본과 observer용 표현을 분리	prompt caching
using/dispose	generator의 모든 종료 경로에서 자원 정리	prefetch lifecycle

프로젝트 구조의 실제 중심은 세 종류의 경계다

디렉터리 명보다 값이 어디서 신뢰를 얻고 효과가 어디서 발생하는지가 중요하다.

01	**Representation boundary**	사용자 입력·메시지·tool schema·SDK event 사이의 변환
02	**Authority boundary**	모델 제안을 permission과 sandbox를 거친 실행으로 승격
03	**Durability boundary**	메모리 state를 transcript·filesystem·subprocess 상태와 합류

UI, MCP, plugin, skill, subagent는 이 경계의 새 producer/consumer다. 따라서 확장 기능을 공부할 때도 “어느 경계를 통과하며 어떤 authority와 durability를 얻는가”를 먼저 묻는다.

리팩터링 단위는 파일이 아니라 불변조건 묶음이다

대형 파일을 작게 나누는 것만으로는 변경 결합이 줄지 않는다. 함께 변해야 하는 계약과 따로 변해야 하는 정책을 구분해야 한다.

현재 결합	분리 후보	보존해야 할 것
query의 stream·recovery·tool orchestration	turn state machine + recovery policy	event 순서와 terminal semantics
Tool의 execution·permission·React rendering	capability/effect/presentation facets	tool identity와 schema 일관성
REPL과 QueryEngine의 lifecycle 중복	headless core + UI/SDK adapters	interactive latency와 resume semantics
session append·projection·resume repair	journal writer + projection/rebuilder	parent chain과 compact/snip semantics
prompt 조립과 cache 규칙	context compiler + cache-key contract	byte/order stability

금지할 단순화

“클린 아키텍처로 다시 짠다”는 목표만으로는 부족하다. 이 코드의 복잡도는 외부 effect·stream·compatibility에서 발생하므로, 경계를 옮길 때 먼저 동작 동등성 oracle이 필요하다.

운영 노력도 코어 코드 안에 들어와 있다

feature gate, telemetry, checkpoint, cost/budget, provider compatibility는 배포 후 학습과 안전한 롤아웃을 위한 제품 메커니즘이다.

관측

turn·stream·tool 지표

request id와 query chain을 결합해 failure funnel을 복원한다.

롤아웃

feature()와 dynamic import

코드 제거·실험군·kill switch가 구조 자체에 영향을 준다.

경제성

prompt cache·compact

token 비용과 latency가 데이터 구조/정렬 순서를 결정한다.

호환성

provider·Node·Windows·legacy alias

동일 core contract를 서로 다른 runtime에 투영한다.

코드 좌표 src/services/api/claude.ts · src/query.ts · src/tools.ts

UI와 generator를 올바른 위치에 놓는다

두 영상은 구현 메커니즘을 이해하는 좋은 렌즈지만 전체 제품 모델을 대신하지 않는다.

영상 렌즈	이 보고서에서의 위치	더 큰 질문
async generator	turn의 progressive observation과 suspension	fallback·tool effect·abort 뒤에도 protocol이 유효한가?
React/Ink UI	interactive projection과 input coordination	정식 상태와 화면 state가 어긋날 때 누가 합류시키는가?

적용 원칙

영상과 같은 맥락의 설명은 해당 메커니즘이 핵심 책임을 어떻게 보존하는지 설명할 때 사용한다. 별도 “기술 소개” 장으로 분리하지 않는다.

한 vertical slice를 끝까지 읽는다

추천 단위는 “prompt 입력”도 “파일 한 개”도 아니다. 하나의 repository effect가 제안되고 확정되며 저장·복구되는 전 구간이다.

1. **Edit**를 고른다: schema와 prompt가 모델에게 무엇을 허용하는지 본다.
2. **validate/checkPermissions**에서 read-before-write, deny rule, path normalization을 추적한다.
3. **call**에서 critical section, patch, LSP/IDE/history 갱신을 본다.
4. **tool_result**가 query state와 다음 API 요청에 어떻게 합류하는지 본다.
5. **abort/fallback/resume**에서 같은 effect가 중복되거나 사라지지 않는지 반례를 만든다.
6. **REPL/SDK**가 동일 사건을 어떻게 다르게 보여 주는지 비교한다.

완료 조건	함수 호출 순서를 말하는 것이 아니라, 어떤 state와 authority가 이동했고 어떤 불변조건이 각 경계에서 보존되었는지 설명할 수 있어야 한다.

함께 공부해야 할 배경지식

프로젝트의 복잡도를 일반화하려면 인접 개념을 코드와 직접 연결해야 한다.

배경지식	연결되는 문제	주의점
effect system / capability security	Tool이 허용 가능한 효과를 기술	TypeScript 타입만으로 OS 권한을 보장하지 않음
optimistic concurrency control	FileEdit stale-write 방지	mtime 검사는 완전한 filesystem transaction이 아님
event sourcing / projections	transcript와 REPL/SDK view	모든 message가 순수 domain event인 것은 아님
structured concurrency	stream·tool·prefetch·abort 수명	AbortSignal은 자동 join/rollback을 제공하지 않음
product-line architecture	feature gate와 build-time DCE	flag가 module boundary를 대신하면 결합이 커짐
cache-key design	system prompt와 tool ordering	semantic 동일성과 byte 동일성이 다름

이후 권들의 질문

다음 보고서들은 관심사 목록이 아니라 이 제품 모델의 네 가지 결정적 문제를 각각 깊게 판다.

VOL. 01

문맥을 프로그램으로 만드는 법

prompt·schema·history·compact·cache가 하나의 실행 입력을 구성한다.

VOL. 02

제안을 코드 효과로 바꾸는 법

Read/Edit/Bash, validation, permission, concurrency, partial effect.

VOL. 03

turn을 복구 가능한 거래로 닫는 법

stream, fallback, terminal, transcript, resume, UI/SDK projection.

VOL. 04

구조를 진화시키는 법

QueryEngine 추출, Tool 분해, feature variability, 운영과 리팩터링 순서.

핵심 질문

“무슨 파일이 호출되는가?”보다 “모델이 제안한 변경이 언제 신뢰를 얻고, 어떤 외부 효과가 되었으며, 실패 뒤 무엇을 진실로 남기는가?”를 묻는다.